

条款 39: 明智而审慎地使用 private 继承

Use private inheritance judiciously.

条款 32 曾经论证过 C++ 如何将 public 继承视为 **is-a** 关系。在那个例子中我们有个继承体系, 其中 class Student 以 public 形式继承 class Person, 于是编译器在必要时刻 (为了让函数调用成功) 将 Students 暗自转换为 Persons。现在我再重复该例的一部分, 并以 private 继承替换 public 继承:

```
class Person { ... };
class Student: private Person { ... };    //这次改用 private 继承
void eat(const Person& p);                //任何人都会吃
void study(const Student& s);              //只有学生才在校学习

Person p;                                  //p 是人
Student s;                                 //s 是学生

eat(p);                                    //没问题, p 是人, 会吃。
eat(s);                                    //错误! 吓, 难道学生不是人?!
```

显然 private 继承并不意味 **is-a** 关系。那么它意味什么?

“哇喔!”你说,“在我们探讨其意义之前,可否先搞清楚其行为。到底 private 继承的行为如何呢?”唔,统御 private 继承的首要规则你刚才已经见过了:如果 classes 之间的继承关系是 private, 编译器不会自动将一个 derived class 对象 (例如 Student) 转换为一个 base class 对象 (例如 Person)。这和 public 继承的情况不同。这也就是为什么通过 s 调用 eat 会失败的原因。第二条规则是,由 private base class 继承而来的所有成员,在 derived class 中都会变成 private 属性,纵使它们在 base class 中原本是 protected 或 public 属性。

够了,现在让我们开始讨论其意义。Private 继承意味 **implemented-in-terms-of** (根据某物实现出)。如果你让 class D 以 private 形式继承 class B, 你的用意是为了采用 class B 内已经备妥的某些特性,不是因为 B 对象和 D 对象存在有任何观念上的关系。private 继承纯粹只是一种实现技术 (这就是为什么继承自一个 private base class 的每样东西在你的 class 内都是 private: 因为它们都只是实现枝节而已)。借用条款 34 提出的术语, private 继承意味只有实现部分被继承,接口部分应略去。如果 D 以 private 形式继承 B, 意思是 D 对象根据 B 对象实现而得,再没有其他意涵了。Private 继承在软件“设计”层面上没有意义,其意义只及于软件实现层面。

Private 继承意味 *is-implemented-in-terms-of* (根据某物实现出), 这个事实有点令人不安, 因为条款 38 才刚指出复合 (composition) 的意义也是这样。你如何在两者之间取舍? 答案很简单: 尽可能使用复合, 必要时才使用 private 继承。何时才是必要? 主要是当 protected 成员和/或 virtual 函数牵扯进来的时候。其实还有一种激进情况, 那是当空间方面的利害关系足以踢翻 private 继承的支柱时。稍后我们再来操这个心, 毕竟它只是一种激进情况。

假设我们的程序涉及 Widgets, 而我们决定应该较好地了解如何使用 Widgets。例如我们不只想要知道 widget 成员函数多么频繁地被调用, 也想知道经过一段时间后调用比例如何变化。要知道, 带有多个执行阶段 (execution phases) 的程序, 可能在不同阶段拥有不同的行为轮廓 (behavioral profiles)。例如编译器在解析 (parsing) 阶段所用的函数, 大大不同于在最优化 (optimization) 和代码生成 (code generation) 阶段所使用的函数。

我们决定修改 widget class, 让它记录每个成员函数的被调用次数。运行期间我们将周期性地审查那份信息, 也许再加上每个 widget 的值, 以及我们需要评估的任何其他数据。为完成这项工作, 我们需要设定某种定时器, 使我们知道收集统计数据的时候是否到了。

我们宁可复用既有代码, 尽量少写新代码, 所以在自己的工具百宝箱中翻箱倒柜, 并且很开心地发现了这个 class:

```
class Timer {
public:
    explicit Timer(int tickFrequency);
    virtual void onTick() const;           //定时器每滴答一次,
    ...                                   //此函数就被自动调用一次。
};
```

这就是我们找到的东西。一个 Timer 对象, 可调整为以我们需要的任何频率滴答前进, 每次滴答就调用某个 virtual 函数。我们可以重新定义那个 virtual 函数, 让后者取出 widget 的当时状态。完美!

为了让 widget 重新定义 Timer 内的 virtual 函数, widget 必须继承自 Timer。但 public 继承在此例并不适当, 因为 widget 并不是个 Timer。是呀, widget 客户总不该能够对着一个 widget 调用 onTick 吧, 因为观念上那并不是 widget 接口的一部分。如果允许那样的调用动作, 很容易造成客户不正确地使用 widget 接口,

那会违反条款 18 的忠告: “让接口容易被正确使用, 不易被误用”。在这里, `public` 继承不是个好策略。

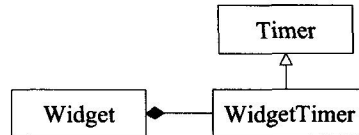
我们必须以 `private` 形式继承 `Timer`:

```
class Widget: private Timer {
private:
    virtual void onTick() const;    //查看 Widget 的数据...等等。
    ...
};
```

藉由 `private` 继承, `Timer` 的 `public onTick` 函数在 `Widget` 内变成 `private`, 而我们重新声明 (定义) 时仍然把它留在那儿。再说一次, 把 `onTick` 放进 `public` 接口内会误导客户端以为他们可以调用它, 那就违反了条款 18。

这是个好设计, 但不值几文钱, 因为 `private` 继承并非绝对必要。如果我们决定以复合 (composition) 取而代之, 是可以的。只要在 `Widget` 内声明一个嵌套式 `private` class, 后者以 `public` 形式继承 `Timer` 并重新定义 `onTick`, 然后放一个这种类型的对象于 `Widget` 内。下面是这种解法的草样:

```
class Widget {
private:
    class WidgetTimer: public Timer {
    public:
        virtual void onTick() const;
        ...
    };
    WidgetTimer timer;
    ...
};
```



这个设计比只使用 `private` 继承要复杂一些些, 因为它同时涉及 `public` 继承和复合, 并导入一个新 class (`WidgetTimer`)。坦白说我展示它主要是为了提醒你, 解决一个设计问题的方法不只一种, 而训练自己思考多种做法是值得的 (看看条款 35)。尽管如此, 我可以想出两个理由, 为什么你可能愿意 (或说应该) 选择这样的 `public` 继承加复合, 而不是选择原先的 `private` 继承设计。

首先, 你或许会想设计 `Widget` 使它得以拥有 `derived classes`, 但同时你可能会想阻止 `derived classes` 重新定义 `onTick`。如果 `Widget` 继承自 `Timer`, 上面的想法就不可能实现, 即使是 `private` 继承也不可能。(还记得吗, 条款 35 曾说 `derived classes` 可以重新定义 `virtual` 函数, 即使它们不得调用它。)但如果 `WidgetTimer` 是 `Widget`

内部的一个 `private` 成员并继承 `Timer`, `Widget` 的 `derived classes` 将无法取用 `WidgetTimer`, 因此无法继承它或重新定义它的 `virtual` 函数。如果你曾经以 `Java` 或 `C#` 编程并怀念“阻止 `derived classes` 重新定义 `virtual` 函数”的能力（也就是 `Java` 的 `final` 和 `C#` 的 `sealed`），现在你知道怎么在 `C++` 中模拟它了。

第二，你或许会想要将 `Widget` 的编译依存性降至最低。如果 `Widget` 继承 `Timer`, 当 `Widget` 被编译时 `Timer` 的定义必须可见，所以定义 `Widget` 的那个文件恐怕必须 `#include Timer.h`。但如果 `WidgetTimer` 移出 `Widget` 之外而 `Widget` 内含指针指向一个 `WidgetTimer`, `Widget` 可以只带着一个简单的 `WidgetTimer` 声明式，不再需要 `#include` 任何与 `Timer` 有关的东西。对大型系统而言，如此的解耦（`decouplings`）可能是重要的措施。关于编译依存性的最小化，详见条款 31。

稍早我曾谈到，`private` 继承主要用于“当一个意欲成为 `derived class` 者想访问一个意欲成为 `base class` 者的 `protected` 成分，或为了重新定义一或多个 `virtual` 函数”。但这时候两个 `classes` 之间的概念关系其实是 `is-implemented-in-terms-of`（根据某物实现出）而非 `is-a`。然而我也说过，有一种激进情况涉及空间最优化，可能会促使你选择“`private` 继承”而不是“继承加复合”。

这个激进情况真是有够激进，只适用于你所处理的 `class` 不带任何数据时。这样的 `classes` 没有 `non-static` 成员变量，没有 `virtual` 函数（因为这种函数的存在会为每个对象带来一个 `vptra`，见条款 7），也没有 `virtual base classes`（因为这样的 `base classes` 也会招致体积上的额外开销，见条款 40）。于是这种所谓的 `empty classes` 对象不使用任何空间，因为没有任何隶属对象的数据需要存储。然而由于技术上的理由，`C++` 裁定凡是独立（非附属）对象都必须有非零大小，所以如果你这样做：

```
class Empty { };                //没有数据，所以其对象应该不使用任何内存

class HoldsAnInt {              //应该只需要一个 int 空间
private:
    int x;
    Empty e;                    //应该不需要任何内存
};
```

你会发现 `sizeof(HoldsAnInt) > sizeof(int)`；喔欧，一个 `Empty` 成员变量竟然要求内存。在大多数编译器中 `sizeof(Empty)` 获得 1，因为面对“大小为零之独

立（非附属）对象”，通常 C++ 官方勒令默默安插一个 char 到空对象内。然而齐位需求（alignment，见条款 50）可能造成编译器为类似 HoldsAnInt 这样的 class 加上一些衬垫（padding），所以有可能 HoldsAnInt 对象不只获得一个 char 大小，也许实际上被放大到足够又存放一个 int。在我试过的所有编译器中，的确有这种情况发生。

但或许你注意到了，我很小心地说“独立（非附属）”对象的大小一定不为零。也就是说，这个约束不适用于 derived class 对象内的 base class 成分，因为它们并非独立（非附属）。如果你继承 Empty，而不是内含一个那种类型的对象：

```
class HoldsAnInt: private Empty {
private:
    int x;
};
```

几乎可以确定 sizeof(HoldsAnInt) == sizeof(int)。这是所谓的 EBO (*empty base optimization*; 空白基类最优化)，我试过的所有编译器都有这样的结果。如果你是一个程序库开发人员，而你的客户非常在意空间，那么值得注意 EBO。另外还值得知道的是，EBO 一般只在单一继承（而非多重继承）下才可行，统治 C++ 对象布局的那些规则通常表示 EBO 无法被施行于“拥有多个 base”的 derived classes 身上。

现实中的 "empty" classes 并不真的是 empty。虽然它们从未拥有 non-static 成员变量，却往往内含 typedefs, enums, static 成员变量，或 non-virtual 函数。STL 就有许多技术用途的 empty classes，其中内含有用的成员（通常是 typedefs），包括 base classes unary_function 和 binary_function，这些是“用户自定义之函数对象”通常会继承的 classes。感谢 EBO 的广泛实践，使这样的继承很少增加 derived classes 的大小。

尽管如此，让我们回到根本。大多数 classes 并非 empty，所以 EBO 很少成为 private 继承的正当理由。更进一步说，大多数继承相当于 **is-a**，这是指 public 继承，不是 private 继承。复合和 private 继承都意味 is-implemented-in-terms-of，但复合比较容易理解，所以无论什么时候，只要可以，你还是应该选择复合。

当你面对“并不存在 **is-a** 关系”的两个 classes，其中一个需要访问另一个的 protected 成员，或需要重新定义其一或多个 virtual 函数，private 继承极有可能成为正统设计策略。即便如此你也已经看到，一个混合了 public 继承和复合的设计，往往能够释出你要的行为，尽管这样的设计有较大的复杂度。“明智而审慎地使用

private 继承”意味，在考虑过所有其他方案之后，如果仍然认为 private 继承是“表现程序内两个 classes 之间的关系”的最佳办法，这才用它。

请记住

- Private 继承意味 is-implemented-in-terms of（根据某物实现出）。它通常比复合（composition）的级别低。但是当 derived class 需要访问 protected base class 的成员，或需要重新定义继承而来的 virtual 函数时，这么设计是合理的。
- 和复合（composition）不同，private 继承可以造成 empty base 最优化。这对致力于“对象尺寸最小化”的程序库开发者而言，可能很重要。

条款 40：明智而审慎地使用多重继承

Use multiple inheritance judiciously.

一旦涉及多重继承（multiple inheritance; MI），C++ 社群便分为两个基本阵营。其中之一认为如果单一继承（single inheritance; SI）是好的，多重继承一定更好。另一派阵营则主张，单一继承是好的，但多重继承不值得拥有（或使用）。本条款的主要目的是带领大家了解多重继承的两个观点。

最先需要认清的一件事是，当 MI 进入设计景框，程序有可能从一个以上的 base classes 继承相同名称（如函数、typedef 等等）。那会导致较多的歧义（ambiguity）机会。例如：

```
class BorrowableItem {           //图书馆允许你借某些东西
public:
    void checkOut( );           //离开时进行检查
    ...
};

class ElectronicGadget {
private:
    bool checkOut( ) const;      //执行自我检测，返回是否测试成功
    ...
};

class MP3Player:                //注意这里的多重继承
    public BorrowableItem,      //(某些图书馆愿意借出 MP3 播放器)
    public ElectronicGadget
{ ... };                       //这里，class 的定义不是我们的关心重点

MP3Player mp;
mp.checkOut( );                //歧义！调用的是哪个 checkOut?
```